VitalFlow — DevOps & Environment Strategy

> Healthcare-grade operational discipline for a Next.js monorepo on Vercel with Supabase. This doc defines how code, secrets, and data move from a developer's laptop to production without surprises.

1. Environment naming convention

Five tiers — three hosted on Vercel + Supabase, two local.

Tier	Purpose	Hosted on	Triggered by	Data realism
`local`	Developer laptop	`supabase start` + `pnpm dev`	Manual	Hand-seeded fixtures
`development`	Shared dev for integration work	Vercel "Development" + `vitalflow-dev` Supabase project	Push to `develop`	Hand-seeded fixtures
`preview`	Per-PR ephemeral deploy	Vercel "Preview" + Supabase branch	Every PR	Branch snapshot of dev
`staging`	Production twin, final gate before production	Vercel "Staging" + `vitalflow-staging` Supabase	Merge to `develop` → auto OR release branch	Scrubbed subset of prod
`production`	Live tenants	Vercel "Production" + `vitalflow-prod` Supabase	Merge to `main` (gated)	Real PHI / BAA signed

Rules:

- **No PHI in `local` / `development` / `preview`.** Scrubbed fixtures only. PHI only appears in `staging` (anonymized subset) and `production`.
- **One Supabase project per tier** (not one project with schemas per tier). Keeps BAA scope, RLS, backups, and blast radius clean.
- Domain convention: `app.vitalflow.health` → prod, `staging.vitalflow.health` → staging, `dev.vitalflow.health` → dev, `*.vitalflow-preview.vercel.app` → previews.

2. Environment variable grouping

Variables are grouped by concern and named consistently. Grouping maps to Vercel env-var scoping and to the `.env.*.example` files in this repo.

Group	Examples	Visibility	Per-env swap?
Runtime	`NODE_ENV`, `LOG_LEVEL`, `VITALFLOW_ENV`	Server + client	Yes
Supabase	`NEXT_PUBLIC_SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`, `SUPABASE_JWT_SECRET`, `SUPABASE_PROJECT_REF`	Split: `NEXT_PUBLIC_*` client; rest server	Yes (separate project per tier)
Auth	`AUTH_SECRET`, `AUTH_COOKIE_DOMAIN`, `AUTH_SESSION_TTL_SECONDS`	Server	Yes (unique secret per tier)
AI	`OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, `AI_DEFAULT_MODEL`	Server	Yes (different keys per tier)
Monetization	`STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY`	Split	Yes — **test keys** in dev/preview/staging; **live keys** only in prod
Notifications	`RESEND_API_KEY`, `TWILIO_*`	Server	Yes — sandbox in non-prod
Integrations	`FHIR_BASE_URL`, `HL7_MLLP_HOST`, `HL7_MLLP_PORT`	Server	Yes — sandbox endpoints in non-prod
Observability	`SENTRY_DSN`, `SENTRY_AUTH_TOKEN`, `NEXT_PUBLIC_POSTHOG_KEY`	Split	Yes — separate projects per tier
Feature flags	`NEXT_PUBLIC_FLAGS_CLIENT_KEY`	Client	Yes
Tenancy	`TENANT_ROUTING_MODE`, `DEFAULT_TENANT_SLUG`	Server	Sometimes

Naming rules:

- `SCREAMING_SNAKE_CASE` always.
- `NEXT_PUBLIC_` prefix is reserved for **non-sensitive** values that are safe to ship in browser bundles.
- No tier suffixes in keys. Scoping is done per Vercel env, not per key.
- Provider prefix first (`STRIPE_`, `SUPABASE_`, `AUTH_`) so grep-by-vendor is one query.

3. Secret handling rules

Non-negotiable:

1. **Never commit secrets.** `.env`, `.env.local`, `.env.*.local` are in .gitignore. Commits are scanned in CI (see security workflow).
2. **Source of truth is Vercel + GitHub Environments**, not the repo. The `.env.*.example` files are contracts listing *which* keys are expected — never their values.
3. **Per-environment secret scopes.** Staging and production have distinct Stripe, Supabase, and observability secrets. Losing a staging key must never expose production.
4. **Quarterly rotation.** All API keys rotate every 90 days. Immediately rotate on:

    - A team member leaves
    - Any suspected compromise
    - Any key accidentally printed to a log or committed
5. **Least-privilege service accounts.** Supabase `service_role` key is scoped to specific server-side code paths; client code uses `anon` key + RLS.
6. **PHI-handling keys are audited.** The database encryption key (Supabase Vault) and the JWT secret are tracked in a compliance log with quarterly review.
7. **No secrets in URLs or logs.** `@vitalflow/shared-utils/logger` redacts common secret paths; verify in code review.
8. **CI runners see secrets only through GitHub Environments**, which requires a required-reviewer approval for the `production` environment.

4. Deployment pipeline

```
┌──────────────┐      PR      ┌──────────────┐     merge     ┌──────────────┐
│   feat/*     │ ──────────▶  │ preview env  │ ──────────▶   │   develop    │
│   branch     │              │ (ephemeral)  │               │   branch     │
└──────────────┘              └──────────────┘               └──────────────┘
        ▲                                                          │
        │ required                                                 │ auto
        │ CI checks                                                ▼
        │                                                  ┌──────────────┐
        │                                                  │ staging deploy │
        │                                                  │ (vitalflow-stg) │
        │                                                  └──────────────┘
        │                                                          │
        │                                                          │
        │                                                  tag vX.Y.Z + PR develop→main
        │                                                          │
        │                                                          ▼
        │                                                  ┌──────────────┐
        │                                                  │ production deploy │
        │                                                  │ (manual approval) │
        │                                                  └──────────────┘
```

Stages:

1. **Feature branch** — developer commits, pushes, opens PR.
2. **CI** — lint, typecheck, unit tests, build, security scan. Required pass before merge.
3. **Preview** — Vercel deploys a preview URL. E2E smoke runs against it. Reviewer can click through.
4. **Merge to `develop`** — auto-deploys to the `development` env + auto-deploys to `staging`. Supabase migrations applied to staging with pre-flight advisor checks.
5. **Release PR** — `develop` → `main`. Requires a tagged version, changelog entry, release checklist, 1 CODEOWNER approval, green CI.

6. **Production deploy** — gated by the `production` GitHub Environment (required reviewer). Applies migrations, deploys apps, runs smoke tests, notifies Slack.

5. Required CI checks

Every PR must pass before merge is allowed:

Check	Workflow	Blocks merge	Notes
`ci / install`	ci.yml	Yes	Frozen lockfile
`ci / lint`	ci.yml	Yes	ESLint + Prettier
`ci / typecheck`	ci.yml	Yes	`tsc --noEmit` across graph
`ci / test`	ci.yml	Yes	Vitest unit + integration
`ci / build`	ci.yml	Yes	`turbo build` with remote cache
`security / audit`	security.yml	Yes	`pnpm audit --audit-level=high`
`security / codeql`	security.yml	Yes	GitHub CodeQL JS+TS
`security / secrets`	security.yml	Yes	gitleaks
`preview / deploy`	preview.yml	No	Comments preview URL on PR
`preview / e2e-smoke`	preview.yml	No	Playwright smoke against preview URL
`db / migration-diff`	ci.yml	Yes	Fails if migration changes miss a repo file

6. Lint / type / test / build gates

Gate contract — each command is idempotent, deterministic, and runnable locally with the same pass/fail as CI:

```
pnpm lint           # ESLint flat + Prettier format:check
pnpm typecheck      # tsc --noEmit across the workspace graph
pnpm test           # Vitest unit tests
pnpm build          # turbo build with cache
pnpm test:e2e       # Playwright (optional locally; required in CI)
```

Standards:

- Zero-warning policy for `lint` on new code. Existing warnings can stay but can't increase.
- Typecheck is strict (`strict: true` , `noUncheckedIndexedAccess: true`).
- Coverage target: 70% line coverage on `services/*` and `packages/*` (enforced once tests exist).

- Build must complete under 6 min on CI (Turborepo remote cache enforced).
- **No `--no-verify`, `--skip-checks`, or `--force`** unless explicitly approved by CODEOWNERS on the PR.

7. Branch protection recommendations

Configured via GitHub rulesets (see .github/rulesets/). Applied to `main` and `develop`, with stricter rules on `main`.

`main` (production)

- Require PR before merge; no direct push.
- Require 1 CODEOWNER review; dismiss stale approvals on new commits.
- Require all status checks to pass: every job under `ci`, `security`.
- Require branches up-to-date before merge.
- Require linear history (squash or rebase merge only; no merge commits).
- Require signed commits.
- Require deployments to succeed on the `staging` environment before merge is allowed.
- No force-push. No branch delete. Admins NOT excluded from rules.
- Block `--no-verify` / hook skipping.

`develop` (integration)

- Require PR before merge; no direct push.
- Require 1 review.
- Require `ci` + `security` checks to pass.
- Linear history preferred.
- No force-push on shared branch.

`release/*` (release candidate)

- Short-lived; cut from `develop`, merged to `main` via tagged PR.
- Same protections as `main`.

`feat/*`, `fix/*`, `chore/*`

- No protections. Owner can force-push freely.

8. Preview deployment workflow

Every PR gets a fully-deployed preview stack that mirrors production topology:

1. **Build** — Vercel triggers a preview build of the unified web app.
2. **Supabase branch** — CI creates a Supabase branch on the `development` project via `supabase branches create`. Branch DBs are ephemeral and inherit schema but not data.
3. **Environment** — the preview app reads preview-scoped env from Vercel, including the branch-specific `SUPABASE_URL`.
4. **Comment** — GitHub Actions posts the preview URL as a sticky PR comment, along with the Supabase branch name.
5. **E2E smoke** — Playwright runs a minimal smoke suite against the preview URL covering all three surfaces (`/`, `/admin`, `/my`).
6. **Cleanup** — on PR close/merge, Supabase branch is deleted (max lifetime 7 days regardless). Vercel previews auto-retire after merge.

Guardrails:

- Preview apps **never** have production Stripe/Twilio/Resend keys. Sandbox providers only.
- Preview env has `VITALFLOW_ENV=preview` so feature flags and banners can surface the env.
- Preview database is **forbidden** from accepting PHI — enforced by column-level constraints on the dev project's `tenants.hipaa_baa_signed` (must be false).

9. Release tagging strategy

Semantic versioning via Changesets. Two versioning tracks:

Apps (user-facing)

- Git tags `v<MAJOR>.<MINOR>.<PATCH>` cut on each production release.
- `MAJOR` — breaking UX (new pricing model, schema-incompatible migration).
- `MINOR` — new features.
- `PATCH` — bug fixes and security patches.
- Each tag is annotated (`git tag -a -s`) and signed.
- GitHub Releases auto-generated from changeset summaries.
- Tag triggers the production deployment workflow.

Internal packages (`@vitalflow/*`)

- Changesets versions `packages/*` and `services/*` independently on merge to `main`.
- `pnpm changeset` is required on any PR that touches a shared package — enforced by changesets-bot.
- Versions published to an internal npm registry (future) or consumed via workspace links today.

Release cadence:

- Production releases shipped **weekly** (Tuesday release window).
- Emergency security patches shipped within 24h via `hotfix/*` branches off `main`.
- Freeze window: no production releases Fri after noon PT through Sun — oncall coverage reasons.

10. Rollback strategy

Target recovery time: ≤ 5 minutes from "critical alert" to "last-good version live."

App rollback (code-only)

1. Vercel UI → target app → "Deployments" tab → pick the last green production deploy → "Promote to Production."
2. Or `vercel rollback <deployment-url> --yes` from the repo root with `VERCEL_TOKEN`.
3. Verify via Sentry error rate + Playwright smoke.

Database rollback (schema or data)

Rule: migrations are forward-only. Never `DOWN` a migration in production.

1. Write a compensating migration (`0014_revert_feature_x.sql`) that undoes the problematic change with proper data-preserving semantics.
2. Apply via the standard deploy pipeline.
3. If the migration itself is catastrophic (data loss), restore from Supabase PITR:
    - Open Supabase Dashboard → Database → Backups → Point-in-time recovery
    - Restore to the minute before the offending deploy
    - Retarget DNS (handled in the runbook)
4. Coordinate with ops; PITR restore takes 15-45 min and disconnects existing sessions.

Feature flags (preferred rollback)

- Any risky change lands behind a flag in `feature_flags` + `feature_flag_overrides`.
- Kill switch = flip the flag. No redeploy required.
- Flag defaults to OFF in production; progressive rollout via `rollout_percent`.

Incident runbook

Every production rollback triggers:

1. PagerDuty incident with appropriate severity.
2. `#vitalflow-incidents` Slack post with alert, symptoms, affected tenants, rollback action.
3. Within 48h: post-mortem doc with timeline, root cause, action items.
4. Changes to prevent recurrence land in `main` before the incident is closed.

Artifacts

- Environment templates: .env.example, .env.staging.example, .env.production.example
- Workflows: ci.yml, security.yml, preview.yml, deploy-staging.yml, deploy-production.yml, release.yml
- Onboarding: contributor-setup.md
- Release: deployment-checklist.md